Search in course

Search

English

B

1. CS 498: Cloud Computing Applications
2. Week 16
3. MP7 Kafka (Optional - Beta Testing)

PreviousNext

Programming Assignment: MP7 Kafka (Optional - Beta Testing)

You have not submitted.

Deadline

Pass this assignment by May 11, 11:59 PM CDT

InstructionsMy submissionsDiscussions

1. Overview

In this MP, you will explore the limitations of synchronous, request-based architectures under unstable and high-volume traffic scenarios and subsequently build a scalable, resilient system using Apache Kafka on AWS. Initially, you will set up a simple Flask consumer on an EC2 instance and simulate uneven traffic to observe how it impacts system stability, leading to system failure.

Then, you'll implement a Kafka-based architecture leveraging AWS Managed Streaming for Apache Kafka (MSK), which can effectively manage spikes in traffic. This includes configuring Kafka producers and consumers, creating topics and partitions, integrating with AWS Lambda for controlled traffic simulation, and storing processed data in HBase running on Amazon EMR. You will also configure monitoring tools (CloudWatch, Prometheus, Grafana) to visualize and manage consumer lag.

After adding additional Kafka consumers and partitions, you will observe improved throughput, better load distribution, and significantly reduced consumer lag, ensuring system resilience during high-volume traffic spikes.

2. Requirements

1. Basic familiarity with AWS services (Lambda, EC2, MSK, EMR, S3)
2. Knowledge of Python programming, Flask web applications, and Kafka concepts (producer, consumer, topic, partition)
3. Ability to SSH into EC2 instances

3. Note

When you are not working on this MP please make sure you are stopping your instances and deleting your MSK cluser and terminating your HBase cluster to stop incurring charges.

4. Procedure

Section 1. Consumer Under High Load

Step 1.1 Build a producer on AWS Lambda

- In this step, you will create an AWS Lambda function that simulates a load test on a consumer endpoint using a predefined template given below. This Lambda will send JSON messages to a /consume endpoint at varying rates over different stages and check the health of the consumer using a /health endpoint.

1

2

3

4

5

6

7

8

9

```
36
37
38
39
40
import json
import time
import urllib3
from concurrent.futures import ThreadPoolExecutor



http = urllib3.PoolManager()
CONSUMER_URL = "http://<consumer_ip:port>/consume"
HEALTH_URL = "http://<consumer_ip:port/health"



STAGES = [
  {"rate": 50, "duration": 10},
  {"rate": 100, "duration": 10},
  {"rate": 200, "duration": 10},
]



def send_request(counter):
  try:
    response = http.request(
```

```
        "POST",
        CONSUMER_URL,
        body=json.dumps({"message": f"load-{counter}"}),
        headers={"Content-Type": "application/json"},
        timeout=2.0
      )
      print(f"[{counter}] Status: {response.status}")
    except Exception as e:
      print(f"[{counter}] ERROR: {str(e)}")



def is_consumer_healthy():
  try:
    r = http.request("GET", HEALTH_URL, timeout=2.0)
    return r.status == 200
  except Exception as e:
    print(f"[HEALTH CHECK FAILED] {str(e)}")
    return False
```

- Replace <consumer_ip:port> with your EC2 instance's IP created in the next step.

Step 1.2 Setup consumer in EC2

- Launch an EC2 instances. We suggest using the latest Amazon Linux AMI as the operating system and the smallest EC2 instance possible (e.g., Nano or Micro) to make the cost of your instances as small as possible (or free.) Attach a security group which allows All traffic in inbound and outbound rules (you can use the same security group in next steps as well). Refer to the link below for the specific steps. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EC2_GetStarted.html

- Ensure you can connect to the EC2 instances using terminal (or the embedded terminal in the EC2 console on your browser) via ssh.
- Now you will implement a flask server which will act as consumer and receive requests from the lambda function created in previous step.
- Open the template given below and complete TODO part. Define the Flask app and the /consume and /health routes.

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

```
# consumer_template.py

"""
Flask Consumer Server Template
------------------------------
This template simulates a backend consumer service that receives POST requests
and processes them with simulated CPU, memory, and time load.
```

```
Your job: Fill in the TODOs to complete the consumer.
"""

from flask import Flask, request
import time

app = Flask(__name__)

#Global list to simulate memory usage
load_list = []



#Define the /consume route
@app.route("/consume", methods=["POST"])
def consume():
    """
    This endpoint simulates a message consumer:
    - Accepts POST requests with JSON payload.
    - Simulates CPU usage (e.g., looping or math operations).
    - Optionally simulates memory usage by appending to a global list.
    - Sleeps for a short time to simulate delay.
    """

    #TODO
    #Parse the incoming JSON data
```

#Simulate CPU work

#Try computing squares in a loop (e.g., [x**2 for x in range(100000)])

#Simulate memory pressure

#Append a large string to the global list to simulate memory load

#Simulate a 100ms delay in processing

#Return a 200 OK response with a status message

- You need to install the required dependencies in the EC2 instance and then start the consumer.
- Test your endpoints by hitting curl commands. Refer the documentation below to learn about curl commands. https://reqbin.com/req/c-1n4ljxb9/curl-get-request-example
- Also install htop in your EC2 instance, you will used htop command to monitor the load.

Step 1.3 Generating traffic and consumer crash

- Make sure your consumer is running in EC2
- Run htop command in your instance to start monitoring the load
- Invoke your lambda function to simulate traffic generation
- You should see CPU and memory shooting up in your EC2 instance and after a few minutes EC2 should stop responding.
- The lambda function will check the health of your consumer periodically and eventually it will not receive ping back from your consumer. At this point, lambda function execution should complete and show status": consumer_unhealthy" message in the output
- You successfully overwhelmed the consumer the with uneven traffic and crashed it

Now that you have observed how a spike in traffic can overwhelm and crash a consumer running on a single EC2 instance, it's clear that synchronous request-based architectures struggle to handle high throughput and sudden load. This brings us to the next stage of the assignment: building a resilient, scalable consumer using Apache Kafka.

Section 2. Configure Kafka to Handle Traffic

In this section you will configure a MSK Kafka cluster which will store messages from producer in partitions. MSK (Managed Streaming for Apache Kafka) is a fully managed service by AWS that allows you to run Apache Kafka without managing the underlying infrastructure. You will then create a lambda function which will generate traffic, create a server which will act as a producer and send the traffic to kafka partitions, create a consumer server on EC2 which will consume the data from kafka partitions and store it in EMR HBase table. Finally you will configure a HBase cluster on EMR which will store the data.

Step 2.1 Configure MSK Kafka Cluster

- Go to the MSK Console and create a cluster.
- Choose Custom create. Enter a name for your cluster. Choose Provisioned as cluster type, Apache Kafka Version 3.6.0, Standard brokers, broker size - kafka.t3.small, Number of zones - 2, Brokers per zone - 1, reduce storage to 1 GiB. Click Next
- Select a VPC and select at least 2 subnets in different AZs. Select a security group with inbound TCP rules configured to allow public access. Click next.
- Select IAM role-based authentication, click next. Select basic monitoring, enable open monitoring with Prometheus, click and next and create cluster The cluster will take around 20-40 minutes to provision.
- **Always delete your cluster when not working on this MP**

Step 2.2 Create a producer lambda function

- Create a S3 bucket and upload following file.

engagement_data

CSV File

- Install requests library in your local folder and create a lambda_function.py file and copy the below code. Zip the content of your local folder (installed libraries and lambda function) and create a lambda function and upload the zip. Refer this

documentation https://docs.aws.amazon.com/lambda/latest/dg/python-package.html

- Give your lambda function read and write access to S3 bucket and increase the lambda timeout to at least 5 minutes. For testing purpose you can increase or decrease the timeout as per your convenience.
- The lambda function loads engagement data from an S3 CSV file, determines traffic type (stable or spiked) based on an input flag, and sends a specified number of messages at controlled intervals to a Kafka producer REST endpoint /produce running on an EC2 instance.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

```
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
import json
import time
import socket
import requests
import csv
```

```
from io import StringIO
from datetime import datetime
import boto3
import requests

# Constants
REGION = 'us-east-1'
TOPIC_NAME = "social_engagement"
BUCKET_NAME = "<your-bucket-name"
CSV_KEY = "<your-csv-file-name>"

def load_csv_from_s3(bucket, key):
    """Reads CSV data from S3 and returns list of dictionaries."""
    try:
        s3 = boto3.client('s3')
        response = s3.get_object(Bucket=bucket, Key=key)
        content = response['Body'].read().decode('utf-8')
        csv_reader = csv.DictReader(StringIO(content))
        return list(csv_reader)
    except Exception as e:
        raise Exception(f"Failed to load CSV from S3: {e}")

def lambda_handler(event, context):
    producer_ip = event.get("producer_ip")
    if not producer_ip:
        raise Exception("Missing 'producer_ip' in event input")
```

```
PRODUCER_URL = f"http://{producer_ip}/produce"


is_stable = event.get("isStableTraffic", True)  # Default: True


# Determine message count and sleep interval based on traffic type
if is_stable:
    message_count = 120
    sleep_interval = 1
```

- The lambda function will be used to send stable and spiked traffic. In production environments, systems often encounter naturally occurring, uneven traffic patterns; however, for the purposes of this assignment and to facilitate controlled experimentation, we simulate such conditions by programmatically generating stable and spiked traffic loads.

Step 2.3 Create a Kafka Client Machine and Create Topic

- Create an IAM role and give it AmazonMSKFullAccess, AmazonEMRFullAccessPolicy_v2, Amazons3ReadOnlyAccess, and an inline policy with following json

```
1
2
3
4
5
6
7
8
9
10
```

```
11
12
13
14
15
16
17
18
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Sid": "KafkaFullWriteAccess",
         "Effect": "Allow",
         "Action": [
            "kafka-cluster:Connect",
            "kafka-cluster:DescribeCluster",
            "kafka-cluster:DescribeTopic",
            "kafka-cluster:WriteData",
            "kafka-cluster:ReadData",
            "kafka-cluster:DescribeGroup"
         ],
         "Resource": "*"
      }
   ]
}
```

- Launch an EC2 Template. Choose an Amazon Linux 2023 AMI and t2.micro instance type. Choose an existing key pair or create a new one if you want SSH access. Attach a security group which allows All traffic in inbound and outbound rules. Attach IAM role created in previous step.
- Run the following commands in your instance. The commands installs java and python, kafka, IAM authentication jar files, creates client.properties files for IAM authentication and install requried libraries.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

```
#!/bin/bash

# Install Java 11 and pip

sudo yum -y install java-11

sudo yum -y install python3-pip


# Install Kafka

cd /home/ec2-user

wget https://archive.apache.org/dist/kafka/3.6.0/kafka_2.13-3.6.0.tgz

tar -xzf kafka_2.13-3.6.0.tgz


# Install MSK IAM Auth jar

wget https://github.com/aws/aws-msk-iam-auth/releases/download/v2.3.0/aws-msk-iam-auth-2.3.0-all.jar

mv aws-msk-iam-auth-2.3.0-all.jar kafka_2.13-3.6.0/libs/


# Configure Kafka client properties

cat <<EOT >> kafka_2.13-3.6.0/config/client.properties

security.protocol=SASL_SSL

sasl.mechanism=AWS_MSK_IAM

sasl.jaas.config=software.amazon.msk.auth.iam.IAMLoginModule required;

sasl.client.callback.handler.class=software.amazon.msk.auth.iam.IAMClientCallbackHandler
```

EOT

Install Python packages

sudo pip3 install boto3 kafka-python aws-msk-iam-sasl-signer-python flask

- Give permission to your EC2 instance to access Kafka Cluster using by running aws configure (you need to give access key, secret key and region for your account). Refer to the documentation below. https://docs.aws.amazon.com/cli/v1/userguide/cli-authentication-user.html
- Now create a kafka topic named social_engagement and 1 partition. Refer to the documentation below. https://docs.aws.amazon.com/msk/latest/developerguide/msk-serverless-create-topic.html

Step 2.4 Create a Flask Server to Route Traffic to Kafka Cluster

- In this step you will build a producer Flask server (template provided below) that accepts engagement events via a POST request and sends them to an MSK Kafka topic using IAM-based authentication. Complete the following steps as part of TODOs in the template.
- To enable this server to communicate with a MSK cluster that uses IAM access control, you will use SASL_OAUTHBEARER mechanism for authentication. Refer to the documentation below for implementation details. https://docs.aws.amazon.com/msk/latest/developerguide/configure-clients-for-iam-access-control.html https://github.com/aws/aws-msk-iam-sasl-signer-python#get-started
- Paste your Kafka broker private endpoint. You can find it on the Cluster summary page, choose View client information.
- Implement the IAM authentication for Kafka and initialize the Kafka Producer
- Implement the /produce route, receives a POST request with a JSON payload containing a topic name and a message, extracts the message key (e.g., user ID), and sends the message to the specified Kafka topic using an IAM-authenticated producer connected to Amazon MSK.

- Implement /health Route, return a JSON with status and machine hostname using socket.gethostname().

Producer

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

```
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
from flask import Flask, request, jsonify
from kafka import KafkaProducer
from kafka.sasl.oauth import AbstractTokenProvider
from aws_msk_iam_sasl_signer import MSKAuthTokenProvider
import json
import socket

# Configuration
# Copy your Kafka broker client below
```

```
REGION = 'us-east-1'
TOPIC_NAME = 'social_engagement'

#TODO: Provide your Kafka broker private endpoint here
KAFKA_BROKERS = [
  "b-1.xxx.amazonaws.com:9098",
   "b-2.xxx.amazonaws.com:9098"
]

# ======================
# TODO: Implement IAM Authentication
# ======================
class MSKTokenProvider(AbstractTokenProvider):
   # code here



# ======================
# TODO: Initialize Kafka Producer
# ======================
producer = KafkaProducer(
   # code here
)

# Flask App
app = Flask(__name__)
```

======================

TODO: Flask Route to Produce Message

======================

@app.route("/produce", methods=["POST"])

def produce():

- Deploy the server in the Kafka client EC2 instance created in previous step. Start the server and test if it is reachable via /health endpoint.
- Now, you will test the /produce endpoint by invoking the producer lambda funcnction created in step 2.2. Use the following test event. { "producer_ip": "<your-EC2-public-ip>:5000", "isStableTraffic": true}
- Your flask server should successfully receive the data from lambda function and send the data to kafka cluster. Run the following command in your kafka client machine to check whether partition is populated with data or not.

1

bin/kafka-run-class.sh kafka.tools.GetOffsetShell --command-config config/client.properties --broker-list <your-kafka-broker-private-endpoint-here> --topic social_engagement

Step 2.5 Setup HBase cluster in AWS EMR

- Go to the EMR Console and create a cluster.
- Choose emr-7.8.0, check HBase from Application Bundle and uncheck rest of the applications.
- Under Cluster configuration, select m4.large as Primary instance type, remove task and core instance group to reduce cost. Ideally A HBase setup will have all three nodes (primary, core and task).
- Select the same VPC as of your EC2 instances. Under Cluster termination select Manually terminate cluster .
- Under Security configuration and EC2 key pair, select a key pair for SSH to the cluster. Otherwise leave it empty if you would use the AWS connect to access your instances.

- Under IAM roles section, click on Create a service role and Create an instance profile. Go ahead and create your cluster.
- After 5-10 mins, an EC2 instance will be provisioned automatically that will be your primary instnace of your HBase database. You will use this instance to access your database.
- Once the instance is up, connect to it via ssh or AWS connect and create a table named post_engagement with column family named engagement. Refer to this documentation to create table. https://docs.aws.amazon.com/emr/latest/ReleaseGuide/emr-hbase-connect.html
- **Always terminate your cluster when not working on this MP**

Step 2.6 Create a Consumer Flask Server

- In this step you will create a consumer flask server which will recevie data from the kafka partition and store it in the HBase database.
- Launch another EC2 instance and run the commands provided in step 2.3 and give permission via aws configure.
- The server will connect to your database using happybase connection. To set up the requried dependencies run following commands

1

2

3

4

5

6

7

```
sudo yum groupinstall "Development Tools" -y

sudo yum install gcc gcc-c++ make -y

sudo yum install python3-devel -y

sudo yum install cmake -y

sudo pip3 install happybase thrift thriftpy2
```

python3 -c "import happybase; print(happybase.__version__)"

- Implement the below template.Paste your. Implement the IAM authentication for Kafka. Replace your Kafka broker private endpoint. Enter your HBase private IP address. Implement the store_in_hbase function to store the receinved message in Hbase table and initialize the Kafka Consumer.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

```
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
from kafka import KafkaConsumer
import json
import happybase  # HBase Client
import time
import socket
from aws_msk_iam_sasl_signer import MSKAuthTokenProvider
from kafka.sasl.oauth import AbstractTokenProvider
```

```
from flask import Flask, jsonify
import threading

# Flask app for health check
app = Flask(__name__)
is_ready = False  # will become True when consumer is fully initialized

@app.route('/health', methods=['GET'])
def health_check():
    if is_ready:
        return jsonify({"status": "UP"}), 200
    return jsonify({"status": "INITIALIZING"}), 503

def start_flask():
    app.run(host='0.0.0.0', port=5000)

# Start Flask in a background thread
threading.Thread(target=start_flask, daemon=True).start()

# ======================
# TODO: Implement IAM Authentication
# ======================
class MSKTokenProvider(AbstractTokenProvider):
    # code here

# TODO: Provide your Kafka broker private endpoint here
```

KAFKA_BROKERS = [

"b-1.xxx.amazonaws.com:9098",

"b-2.xxx.amazonaws.com:9098"

]

TOPIC = "social_engagement"

TODO: HBase Configuration

- Now you will test your setup end to end. Start your both servers and invoke lambda function, your consumer should successfully receive and store the data in the HBase database now.
- Use following script to check the consumer lag. You will see that negligible lag is being generated because consumer is consuming message at the same speed as producer is producing them.

1

bin/kafka-consumer-groups.sh --bootstrap-server <your-kafka-broker-private-endpoint here> --command-config config/client.properties --describe --group engagement_analytics_group

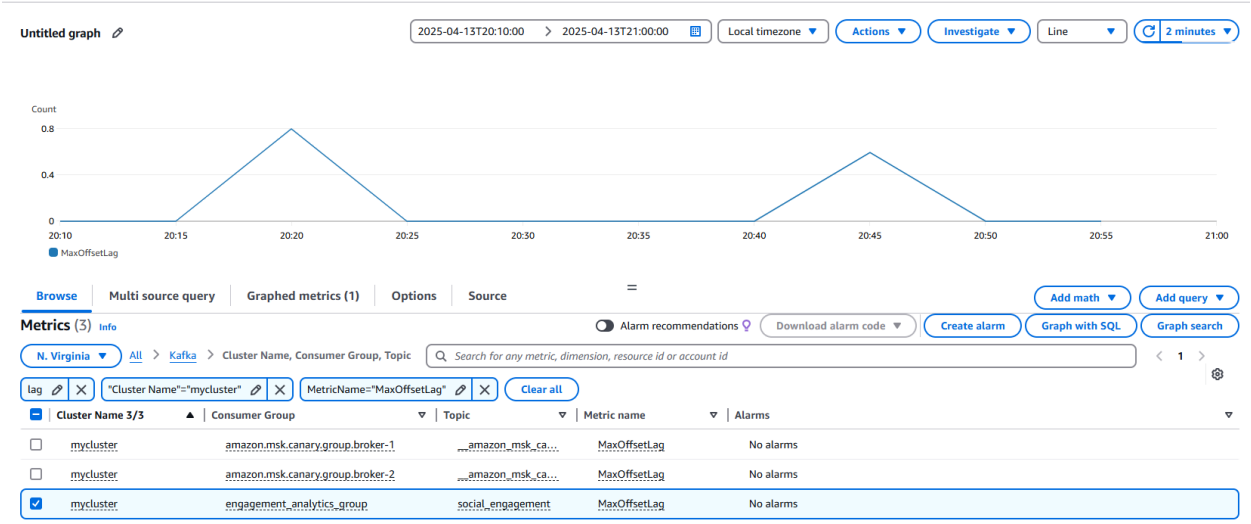
Section 3. Visualize Consumer Lag

In this section you will setup CloudWatch and later Prometheus with Grafana to visualize the lag metrics.

Step 3.1 Setup CloudWatch

- Go to CloudWatch and select all metrics, search for lag and select Kafka > Cluster Name, Consumer Group, Topic
- Filter your cluster name (social_engagement), topic and select MaxOffSetLag. You should be able to see the graph now.
- Invoke your lambda function with the following test event. { "producer_ip": "<your-EC2-public-ip>:5000", "isStableTraffic": true} . This will send stable traffic to your Kafka and consumer should be able to handle this traffic and hence no lag will be produced.

- After few minutes of invoking the lambda function you will see a similar graph (not identical) like this in CloudWatch indicating almost no consumer lag.



- If you observe, CloudWatch introduces a delay in reporting Kafka consumer lag metrics, making it unsuitable for real-time monitoring or quick detection of processing bottlenecks.
- Hence, now you will setup Prometheus + Grafana for near real-time lag monitoring. Prometheus scrapes metrics exposed by JMX Exporter (for Kafka MSK JVM metrics) and Node Exporter (for EC2 system metrics) as defined in prometheus.yml (scrape config) and targets.json (target endpoints); Grafana visualizes this data using dashboards by querying Prometheus.

Step 3.2: Setup Prometheus

- Launch another EC2 instance with **t2.small** instance type
- Run following commands in your instance to install prometheus.

1

2

3

4

wget https://github.com/prometheus/prometheus/releases/download/v2.51.2/prometheus-2.51.2.linux-amd64.tar.gz

tar -xvf prometheus-2.51.2.linux-amd64.tar.gz

mv prometheus-2.51.2.linux-amd64 prometheus

- Now go inside prometheus folder and edit prometheus.yml file and create a targets.json file with the scripts given below. Paste your kafka client broker id in targets.json file.

prometheus.yml

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

```
# prometheus. yml file copy from next line
# file: prometheus.yml
# my global config
global:
  scrape_interval:    60s

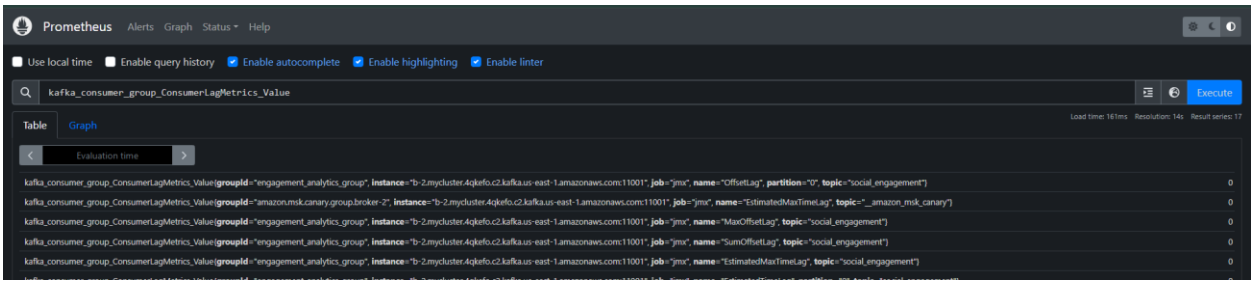

# A scrape configuration containing exactly one endpoint to scrape:
```

```
# Here it's Prometheus itself.
scrape_configs:
 # The job name is added as a label `job=<job_name>` to any timeseries scraped from this config.
 - job_name: 'prometheus'
  static_configs:
  # 9090 is the prometheus server port
  - targets: ['localhost:9090']
 - job_name: 'broker'
  file_sd_configs:
  - files:
   - 'targets.json'


#target.json
#replace your kafka broker private endpoint, remove port 9098 from endpoint
#copy from next line
[
 {
  "labels": {
   "job": "jmx"
  },
  "targets": [
   "<your-kafka-broker-client-here>",
   "<your-kafka-broker-client-here>"
  ]
 },
```

```
{
 "labels": {
  "job": "node"
 },
 "targets": [
  "<your-kafka-broker-client-here>",
  "<your-kafka-broker-client-here>"
 ]
```

- Start your prometheus server by using this command: ./prometheus --config.file=prometheus.yml &
- Access prometheus console by accessing <prometheus-ec2-public-ip>:9090 in your web browser.
- In the prometheus console search for kafka_consumer_group_ConsumerLagMetrics_Value. You should see some results like this.



Step 3.3 Setup Grafana

- Within the same EC2 instance, run following command to setup grafana

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

```
sudo rpm --import https://packages.grafana.com/gpg.key

echo "[grafana]
name=Grafana Repository
baseurl=https://packages.grafana.com/oss/rpm
repo_gpgcheck=1
enabled=1
gpgcheck=1
gpgkey=https://packages.grafana.com/gpg.key" | sudo tee /etc/yum.repos.d/grafana.repo

sudo yum install -y grafana
```

#Enable grafana server

sudo systemctl enable grafana-server

#start grafana server

sudo systemctl start grafana-server

#check status of grafana server (optional)

sudo systemctl status grafana-server

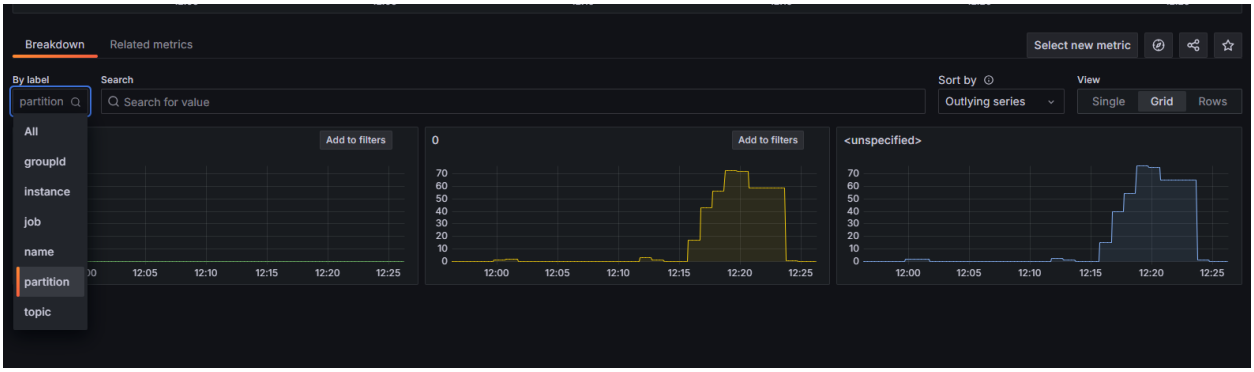
- To access grafana dashboard access <your-ec2-public-ip>:3000 in your web browser. Enter admin for both username and password. Skip password update.
- Configure prometheus as data source in grafana. Select No Authentication and 5s for Scrape interval. Follow the steps given in the docs below. https://grafana.com/docs/grafana/latest/datasources/prometheus/configure-prometheus-data-source/
- Follow the steps in the following document to visualize consumer lag, search for metric kafka_consumer_group_ConsumerLagMetrics_Value. Setup Drilldown metrics

Step 3.4 Send Stable Traffic and Observe Lag

- You will now send stable traffic from your producer and observe the consumer lag in Grafana. A total of 120 messages would be sent and your HBase table should also contains 120 rows after the lambda function stops its exectution.
- After few minutes of your execution, you should see a similar (not identical) graph in Grafana. This shows that at stable traffic, your consumer is able to traffic handle and negligible consumer lag is being generated.

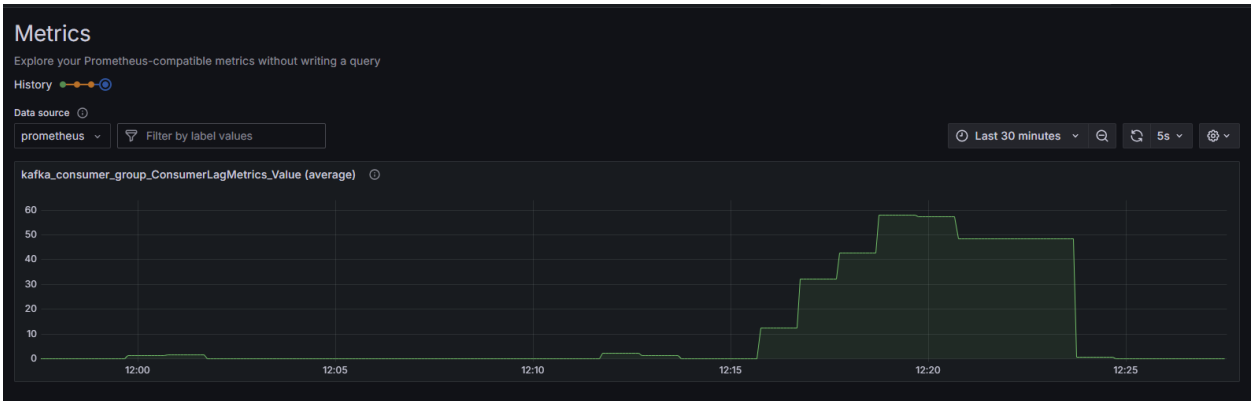


- You can further see the lag metrics by partition, name, instance, etc in the breakdown section below the above graph.



Step 3.5 Send Spiked Traffic and Observe Lag

- In this step, you will send spiked traffic from the lambda function and observe how the lag changes under load. Change the isStableTraffic key to false in your test event to send the spiked traffic. This will start sending a total 0f 500 messages at twice the rate at which consumer is consuming the messages.
- In this scenario, consumer will start lagging and eventually consumer lag will increase as shown in Grafana below.



- In the next sections, you will mitigate the lag by scaling consumers and partitions.

Section 4. Add Another Consumer to Reduce Lag

- In this step you will add another consumer to reduce the lag.
- Follow steps 2.6 and Create another consumer EC2 instance.
- Now with your producer server and two consumers running, once again send a spiked traffic through your lambda function and observe the lag.
- Run the script for checking consumer lag and you would you would observe that only one consumer is consuming messages from the partition . And the lag is also being generate despite of another consumer.
- Run the following script to see partition assigment to each consumer. You will that one of the consumer has 0 parition.

1

bin/kafka-consumer-groups.sh --bootstrap-server <your-kafka-broker-private-endpoint-here> --command-config config/client.properties --describe --
group engagement_analytics_group --members

- So what is really happening here? The answer is Kafka assigns the partitions of a topic to the consumer in a consumer group, so that each partition is consumed by exactly one consumer in the consumer group. Kafka guarantees that a message is only ever read by a single consumer in the consumer group . Hence another consumer stays idle.
- In order to make another consumer active, in the next section, you will increase the partition within your kafka cluster.

Section 5. Increase Partition

- Run the following script to increase your partition

1

bin/kafka-topics.sh --command-config config/client.properties --bootstrap-server <your-kafka-broker-private-endpoint-here> --alter --topic social_engagement --partitions 4

- Send the spiked tarffic again and observe the lag via Grafana and the scripts to check the lag . You should now see that both consumer is consuming messages from partition and equal partitions are assigned to both consumers. Graph will also show minimal lag now highlighted in red.



- You have successfully mitigated the consumer lag by scaling both consumers and partitions.

Section 6 Submission

- Download the submit.py file or copy from the template below

1

2

3

4

5

6

7

8

9

10

11

12

13

14

```
import requests
import json

# === Fill in your identity info ===
YOUR_EMAIL = "" # <Your Coursera email>
YOUR_SECRET = "" # <Secret token provided to you>

# === Fill in your infrastructure values ===
HBASE_TABLE = "post_engagement"
HBASE_MASTER_IP = "" #Enter your HBase Public IP
PRODUCER_IP = "public-ip:5000" #Enter your Producer Public IP
CONSUMER1_IP = "public-ip:5000" #Enter your Consumer 1 Public IP
CONSUMER2_IP = "public-ip:5000" #Enter your Consumer 2 Public IP

# === API Gateway URL ===
API_GATEWAY_URL = "https://2e37erweu6.execute-api.us-east-1.amazonaws.com/PROD/"

# === Prepare payload ===
input_payload = {
    "submitterEmail": YOUR_EMAIL,
    "secret": YOUR_SECRET,
    "hbase_table": HBASE_TABLE,
    "hbase_master_ip": HBASE_MASTER_IP,
    "producer_ip": PRODUCER_IP,
    "consumer1_ip": CONSUMER1_IP,
```

```
    "consumer2_ip": CONSUMER2_IP
}

# === Final payload for API Gateway ===
execution_payload = {
    "input": input_payload,
    "stateMachineArn": "arn:aws:states:us-east-1:913708708374:stateMachine:mp7_kafka_grader"
}

# === Send the POST request to the token validation Lambda API ===
try:
    response = requests.post(API_GATEWAY_URL, json={
    "body": json.dumps(execution_payload)  # Wrap payload in "body"
})
    print("Status:", response.status_code, response.reason)
```

submit

PY File

- Enter your infrastructure details in the submission file.
- Before submitting ensure producer server is running, both consumer servers are running, topic social_engagement is created, parition is equal to 4, post_engagement table is present in your HBase cluster
- Then run submit.py on your local machine. You will receive the autograder's feedback for the current submission, and your grade on Coursera will be updated automatically. Note that the grading process may take 5-10 minutes. Monitor the

submissions section actively for any failure feedback. Intermediate failures will be promptly updated to minimize debugging time.

- Also, **before every submission, make sure updating your submission token,** since the valid time for a token is only 30 minutes.
- **Stop/Delete your instances, delete your MSK cluser and terminate your HBase cluster to stop incurring charges.**

How to submit

Copy the token below and run the submission script included in the assignment download. When prompted, use your email address **fridkis2@illinois.edu**.

Generate new token

Your submission token is unique to you and should not be shared with anyone. You may submit as many times as you like.

Like

Dislike

Report an issue